

## Table of Contents

|                                        |  |
|----------------------------------------|--|
| 1. 定义 .....                            |  |
| 2. 使用 .....                            |  |
| 3. AQS .....                           |  |
| 3.1 资源共享模式 .....                       |  |
| 3.2 AQS的主要方法源码解析 .....                 |  |
| 3.2.1 获取资源 .....                       |  |
| 3.2.2 释放资源 .....                       |  |
| 3.2.3 超时释放资源 .....                     |  |
| 3.3 ReentrantLock (重入锁) .....          |  |
| 3.4 ReentrantReadWriteLock (读写锁) ..... |  |
| 3.5 StampedLock .....                  |  |
| 3.6 Condition 接口 .....                 |  |

## 1. 定义

在 Lock 接口出现之前，Java 程序是靠 synchronized 关键字实现锁功能的，而 Java SE 5之后，并发包中新增了 Lock 接口（以及相关实现类）用来实现锁功能，它提供了与synchronized 关键字类似的同步功能,只是在使用时需要显式地获取和释放锁。

虽然它缺少了（通过 synchronized 块或者方法所提供的）隐式获取释放锁的便捷性，但是却拥有了锁获取与释放的可操作性、可中断的获取锁以及超时获取锁等多种 synchronized 关键字所不具备的同步特性。

Lock也是维护了一个锁（state），和一个等待队列(AQS)，这也是Lock在底层实现的两个核心元素。AQS队列解决了线程同步的问题，volatile定义的锁状态解决保证了线程对于临界区代码访问的互斥，并且解决了各个线程对于锁状态的可见性问题。

Lock 是一个接口，它定义了锁获取和释放的基本操作

| 方法名称                                                                               | 描 述                                                                                             |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>void lock()</code>                                                           | 获取锁，调用该方法当前线程将会获取锁，当锁获得后，从该方法返回                                                                 |
| <code>void lockInterruptibly() throws InterruptedException</code>                  | 可中断地获取锁，和 <code>lock()</code> 方法的不同之处在于该方法会响应中断，即在锁的获取中可以中断当前线程                                 |
| <code>boolean tryLock()</code>                                                     | 尝试非阻塞的获取锁，调用该方法后立刻返回，如果能够获取则返回 <code>true</code> ，否则返回 <code>false</code>                       |
| <code>boolean tryLock(long time, TimeUnit unit) throws InterruptedException</code> | 超时的获取锁，当前线程在以下 3 种情况下会返回：<br>①当前线程在超时时间内获得了锁<br>②当前线程在超时时间内被中断<br>③超时时间结束，返回 <code>false</code> |
| <code>void unlock()</code>                                                         | 释放锁                                                                                             |
| <code>Condition newCondition()</code>                                              | 获取等待通知组件，该组件和当前的锁绑定，当前线程只有获得了锁，才能调用该组件的 <code>wait()</code> 方法，而调用后，当前线程将释放锁                    |

## 2. 使用

一般使用Lock会使用它的实现类, java中常用的就是ReentrantLock 类

```
Lock lock = new ReentrantLock();
lock.lock();
try {
    // do some thing
} finally {
    lock.unlock();
}
```

不要将获取锁的过程写在 try 块中，因为如果在获取锁（自定义锁的实现）时发生了异常，异常抛出的同时，也会导致锁无故释放。

关键字synchronized与wait()和notify()/notifyAll()方法相结合可以实现等待/通知模式，类ReentrantLock也可以实现同样的功能，但需要借助于Condition对象

condition需要获得锁后使用

```
private Lock lock = new ReentrantLock();
private Condition condition = lock.newCondition();
```

一把锁可以生成多个condition, (变量各自控制各自线程的等待和唤起)

当调用 await()方法后，当前线程会释放锁并在此等待，而其他线程调用 Condition 对象的 signal() 方法，通知当前线程后，当前线程才从 await()方法返回，并且在返回前已经获取了锁

### 3. AQS

抽象队列同步器 AbstractQueuedSynchronizer（以下简称同步器），是用来构建锁或者其他同步组件的基础框架，它使用了一个 int 成员变量表示同步状态，通过内置的 FIFO 队列来完成资源获取线程的排队工作。

AQS的主要使用方式是继承它作为一个内部辅助类实现同步原语，它可以简化你的并发工具的内部实现，屏蔽同步状态管理、线程的排队、等待与唤醒等底层操作。AQS设计基于模板方法模式，开发者需要继承同步器并且重写指定的方法，将其组合在并发组件的实现中，调用同步器的模板方法，模板方法会调用使用者重写的方法。

使用AQS能简单且高效地构造出应用广泛的同步器，比如我们提到的ReentrantLock，Semaphore，ReentrantReadWriteLock，SynchronousQueue，FutureTask等等皆是基于AQS的。

- ReentrantLock: 使用了AQS的独占获取和释放,用state变量记录某个线程获取独占锁的次数,获取锁时+1，释放锁时-1，在获取时会校验线程是否可以获取锁。
- Semaphore: 使用了AQS的共享获取和释放，用state变量作为计数器，只有在大于0时允许线程进入。获取锁时-1，释放锁时+1。
- CountdownLatch: 使用了AQS的共享获取和释放，用state变量作为计数器，在初始化时指定。只要state还大于0，获取共享锁会因为失败而阻塞，直到计数器的值为0时，共享锁才允许获取，所有等待线程会被逐一唤醒。

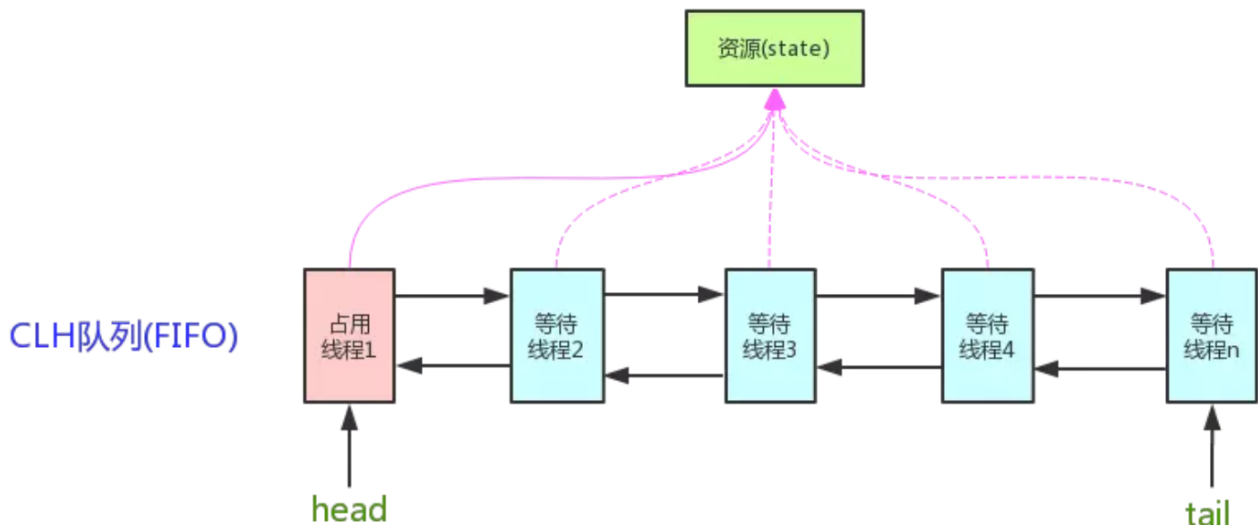
当然，我们自己也能利用AQS非常轻松容易地构造出符合我们自己需求的同步器，只要子类实现它的几个 `protected` 方法就可以了，

AQS内部使用了一个volatile的变量state来作为资源的标识。同时定义了几个获取和改变state的protected方法，子类可以覆盖这些方法来实现自己的逻辑：

```
getState()  
setState()  
compareAndSetState()
```

这三种叫做均是原子操作，其中compareAndSetState的实现依赖于Unsafe的compareAndSwapInt()方法。

而AQS类本身实现的是一些排队和阻塞的机制，比如具体线程等待队列的维护（如获取资源失败入队/唤醒出队等）。它内部使用了一个先进先出（FIFO）的双端队列，并使用了两个指针head和tail用于标识队列的头部和尾部。其数据结构如图：



## AQS如何防止内存泄露

AQS维护了一个FIFO队列，AQS在无竞争条件下，甚至都不会new出head和tail节点。

线程成功获取锁时设置head节点的方法为setHead，由于头节点的thread并不重要，此时会置node的thread和prev为null，

完了之后还会置原先head也就是线程对应node的前驱的next为null，从而实现队首元素的安全移出。

而在取消节点时，也会令node.thread = null，在node不为tail的情况下，会使node.next = node（之所以这样也是为了isOnSyncQueue实现更加简洁）

虽然不是很懂....

[AbstractQueuedSynchronizer源码解读 - 活在夢裡 - 博客园 \(cnblogs.com\)](http://cnblogs.com)

大致思路:

AQS内部维护一个CLH队列来管理锁。

1. 线程会首先尝试获取锁，如果失败，则将当前线程以及等待状态等信息包成一个Node节点加到同步队列里。
2. 接着会不断循环尝试获取锁（条件是当前节点为head的直接后继才会尝试），如果失败则会阻塞自己，直至被唤醒；
3. 而当持有锁的线程释放锁时，会唤醒队列中的后继线程。

总结: 通过一个队列来让线程们排队执行,就做到了线程安全, 然后通过一个状态来做标识(例如ReentrantLock用这个状态来表示加锁次数),

在添加队列时需要获得(自己的)锁才能添加队列, 加到队列后让当前线程阻塞(park()函数),每次释放锁时,就把队列中的下一个线程启动(unpark()函数)

### 线程中断:

在一个线程正常结束之前，如果被强制终止，那么就有可能造成一些比较严重的后果，设想一下如果现在有一个线程持有同步锁，然后在没有释放锁资源的情况下被强制休眠，那么这就造成了其他线程无法访问同步代码块。因此我们可以看到在 Java 中类似 `Thread#stop()` 方法被标为 `@Deprecated`。针对上述情况，我们不能直接将线程给终止掉，但有时又必须将让线程停止运行某些代码，那么此时我们必须有一种机制让线程知道它该停止了。此时可以通过 `Thread#interrupt()` 给线程该线程一个标志位，让该线程自己决定该怎么办。

[Java线程中断\(Interrupt\)与阻塞\(park\)的区别 - 周二鸭 - 博客园 \(cnblogs.com\)](#)

[Thread.sleep、synchronized、LockSupport.park的线程阻塞区别 - 知乎 \(zhihu.com\)](#)

## 3.1 资源共享模式

资源有两种共享模式，或者说两种同步方式：

- 独占模式（Exclusive）：资源是独占的，一次只能一个线程获取。如ReentrantLock。
- 共享模式（Share）：同时可以被多个线程获取，具体的资源个数可以通过参数指定。如Semaphore/CountDownLatch。

就是共享锁和独占锁

一般情况下，子类只需要根据需求实现其中一种模式，当然也有同时实现两种模式的同步类，如 `ReadWriteLock`。

前面说到,AQS通过队列来控制获取线程工作, 这个队列是先进先出的双向队列, 两种模式下都用了这个队列, 通过 `Node` 类来控制, 它下面有 前驱节点,后驱节点, 下一个等待节点等等属性.

注意: 通过Node我们可以实现两个队列, 一是通过prev和next实现CLH队列(线程同步队列, 双向队列), 二是nextWaiter实现Condition条件上的等待线程队列(单向队列), 这个Condition主要用在ReentrantLock类中。

## 3.2 AQS的主要方法源码解析

AQS的设计是基于**模板方法模式**的, 它有一些方法必须要子类去实现的, 它们主要有:

- `isHeldExclusively()`: 该线程是否正在独占资源。只有用到condition才需要去实现它。
- `tryAcquire(int)`: 独占方式。尝试获取资源, 成功则返回true, 失败则返回false。
- `tryRelease(int)`: 独占方式。尝试释放资源, 成功则返回true, 失败则返回false。
- `tryAcquireShared(int)`: 共享方式。尝试获取资源。负数表示失败; 0表示成功, 但没有剩余可用资源; 正数表示成功, 且有剩余资源。
- `tryReleaseShared(int)`: 共享方式。尝试释放资源, 如果释放后允许唤醒后续等待结点返回true, 否则返回false。

这些方法虽然都是 `protected` 方法, 但是它们并没有在AQS具体实现, 而是直接抛出异常(这里不使用抽象方法的目的是: 避免强迫子类中把所有的抽象方法都实现一遍, 减少无用功, 这样子类只需要实现自己关心的抽象方法即可

比如 Semaphore 只需要实现 `tryAcquire` 方法而不用实现其余不需要用到的模版方法):

```
protected boolean tryAcquire(int arg) {  
    throw new UnsupportedOperationException();  
}
```

首先看一下AQS中的嵌套类Node的定义。

```
static final class Node {

    /**
     * 用于标记一个节点在共享模式下等待
     */
    static final Node SHARED = new Node();

    /**
     * 用于标记一个节点在独占模式下等待
     */
    static final Node EXCLUSIVE = null;

    /**
     * 等待状态：取消
     */
    static final int CANCELLED = 1;

    /**
     * 等待状态：通知
     */
    static final int SIGNAL = -1;

    /**
     * 等待状态：条件等待
     */
    static final int CONDITION = -2;

    /**
     * 等待状态：传播
     */
    static final int PROPAGATE = -3;

    /**
     * 等待状态
     */
    volatile int waitStatus;

    /**
     * 前驱节点
     */
    volatile Node prev;

    /**
     * 后继节点
     */
    volatile Node next;

    /**
     * 节点对应的线程
     */
}
```

```

volatile Thread thread;

/**
 * 等待队列中的后继节点
 */
Node nextWaiter;

/**
 * 当前节点是否处于共享模式等待
 */
final boolean isShared() {
    return nextWaiter == SHARED;
}

/**
 * 获取前驱节点，如果为空的话抛出空指针异常
 */
final Node predecessor() throws NullPointerException {
    Node p = prev;
    if (p == null) {
        throw new NullPointerException();
    } else {
        return p;
    }
}

Node() {
}

/**
 * addWaiter会调用此构造函数
 */
Node(Thread thread, Node mode) {
    this.nextWaiter = mode;
    this.thread = thread;
}

/**
 * Condition会用到此构造函数
 */
Node(Thread thread, int waitStatus) {
    this.waitStatus = waitStatus;
    this.thread = thread;
}
}

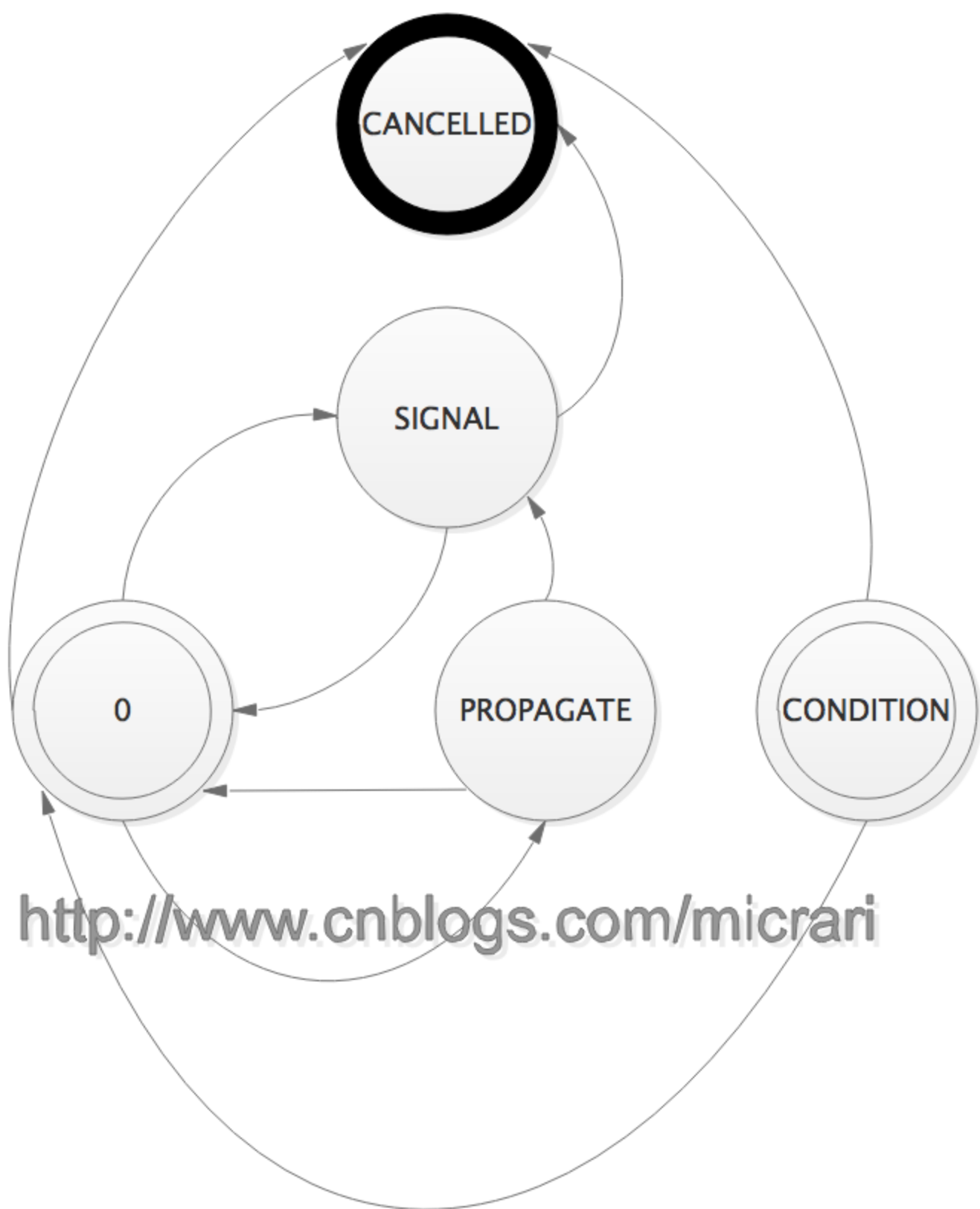
```

这里有必要专门梳理一下节点等待状态的定义，因为AQS源码中有大量的状态判断与跃迁。



| 值                 | 描述                                                             |
|-------------------|----------------------------------------------------------------|
| CANCELLED<br>(1)  | 当前线程因为超时或者中断被取消。这是一个终结态，也就是状态到此为止。                             |
| SIGNAL (-1)       | 当前线程的后继线程被阻塞或者即将被阻塞，当前线程释放锁或者取消后需要唤醒后继线程。这个状态一般都是后继线程来设置前驱节点的。 |
| CONDITION<br>(-2) | 当前线程在condition队列中。                                             |
| PROPAGATE<br>(-3) | 用于将唤醒后继线程传递下去，这个状态的引入是为了完善和增强共享锁的唤醒机制。在一个节点成为头节点之前，是不会跃迁为此状态的  |
| 0                 | 表示无状态。                                                         |

对于分析AQS中不涉及 `ConditionObject` 部分的代码，可以认为队列中的节点状态只会是 CANCELLED, SIGNAL, PROPAGATE, 0这几种情况。



以下仅讲述独占锁的方式

### 3.2.1 获取资源

获取资源的入口是`acquire(int arg)`方法。`arg`是要获取的资源的个数，在独占模式下始终为1。我们先来看看这个方法的逻辑：

```
public final void acquire(int arg) {  
    if (!tryAcquire(arg) &&  
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))  
        selfInterrupt();  
}
```

首先调用`tryAcquire(arg)`尝试去获取资源。前面提到了这个方法是在子类具体实现的。

如果获取资源失败，就通过`addWaiter(Node.EXCLUSIVE)`方法把这个线程插入到等待队列中。其中传入的参数代表要插入的Node是独占式的。

```

private Node addWaiter(Node mode) {
    // 生成该线程对应的Node节点
    Node node = new Node(Thread.currentThread(), mode);
    // 将Node插入队列中
    Node pred = tail;
    if (pred != null) {
        node.prev = pred;
        // 使用CAS尝试，如果成功就返回
        if (compareAndSetTail(pred, node)) {
            pred.next = node;
            return node;
        }
    }
    // 如果等待队列为空或者上述CAS失败，再自旋CAS插入
    enq(node);
    return node;
}

/**
 * 通过循环+CAS在队列中成功插入一个节点后返回。
 */
private Node enq(final Node node) {
    for (;;) {
        Node t = tail;
        // 初始化head和tail
        if (t == null) {
            // 不知道为什么要单独new一个节点,而不是用入参, 而且它们表示的资源共享模式也不一样????????
            if (compareAndSetHead(new Node()))
                tail = head;
        } else {
            /*
             * AQS的精妙就是体现在很多细节的代码, 比如需要用CAS往队尾里增加一个元素
             * 此处的else分支是先在CAS的if前设置node.prev = t, 而不是在CAS成功之后再设置。
             * 一方面是基于CAS的双向链表插入目前没有完美的解决方案, 另一方面这样子做的好处是:
             * 保证每时每刻tail.prev都不会是一个null值, 否则如果node.prev = t
             * 放在下面if的里面, 会导致一个瞬间tail.prev = null, 这样会使得队列不完整。
             *
             * 这里的双向队列添加尾节点其实有三步, 1. 把node的前驱节点挂在尾节点上; 2. 把tail标识改到node上; 3. 把尾节点的后驱节点连上node
             * 如果把node.prev = t 放到cas中, 相当于先执行了第二步, 就会出现短暂的队列不完整 (队列tail不见了)
             */
            node.prev = t;
            // CAS设置tail为node, 成功后把老的tail也就是t连接到node。
            if (compareAndSetTail(t, node)) {
                t.next = node;
                return t;
            }
        }
    }
}

```

```

    }
}
}
}

```

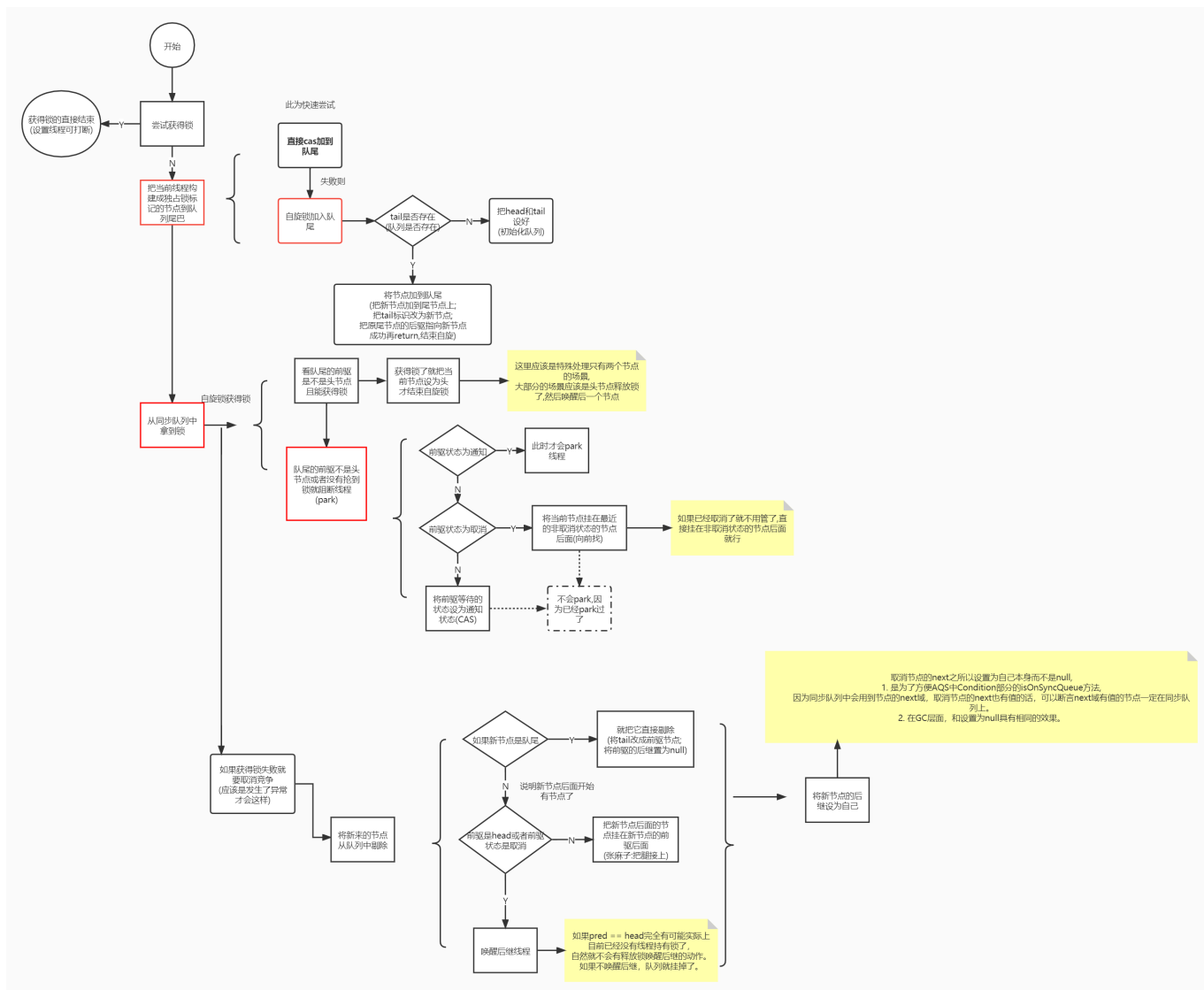
上面的两个函数比较好理解，就是在队列的尾部插入新的Node节点，但是需要注意的是由于AQS中会存在多个线程同时争夺资源的情况，因此肯定会出现多个线程同时插入节点的操作，在这里是通过CAS自旋的方式保证了操作的线程安全性。

OK，现在回到最开始的acquire(int arg)方法。现在通过addWaiter方法，已经把一个Node放到等待队列尾部了。而处于等待队列的结点是从头结点一个一个去获取资源的。具体的实现我们来看看acquireQueued方法

```

final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        // 自旋
        for (;;) {
            final Node p = node.predecessor();
            // 如果node的前驱结点p是head，表示node是第二个结点，就可以尝试去获取资源了
            if (p == head && tryAcquire(arg)) {
                // 拿到资源后，将head指向该结点。
                // 所以head所指的结点，就是当前获取到资源的那个结点或null。
                setHead(node);
                p.next = null; // help GC
                failed = false;
                return interrupted;
            }
            // 如果自己可以休息了，就进入waiting状态，直到被unpark()
            if (shouldParkAfterFailedAcquire(p, node) && parkAndCheckInterrupt())
                interrupted = true;
        }
    } finally {
        if (failed)
            // 发生异常了，需要取消这个node获得锁
            cancelAcquire(node);
    }
}

```



获取锁的大概通用流程如下：

线程会首先尝试获取锁，如果失败，则将当前线程以及等待状态等信息包成一个Node结点加到同步队列里。接着会不断循环尝试获取锁（获取锁的条件是当前结点为head的直接后继才会尝试），如果失败则会尝试阻塞自己（阻塞的条件是当前节结点的前驱结点是SIGNAL状态,所以大部分节点都处于自旋状态），阻塞后将不会执行后续代码，直至被唤醒；当持有锁的线程释放锁时，会唤醒队列中的后继线程，或者阻塞的线程被中断或者时间到了，那么阻塞的线程也会被唤醒。

这里parkAndCheckInterrupt方法内部使用到了LockSupport.park(this)，顺便简单介绍一下park。

LockSupport类是Java 6 引入的一个类，提供了基本的线程同步原语。LockSupport实际上是调用了Unsafe类里的函数，归结到Unsafe里，只有两个函数：

- park(boolean isAbsolute, long time)：阻塞当前线程
- unpark(Thread jthread)：使给定的线程停止阻塞

所以**结点进入等待队列后，是调用park使它进入阻塞状态的。只有头结点的线程是处于活跃状态的。**

当然，获取资源的方法除了acquire外，还有以下三个：

- acquireInterruptibly：申请可中断的资源（独占模式）
- acquireShared：申请共享模式的资源
- acquireSharedInterruptibly：申请可中断的资源（共享模式）

可中断的意思是，在线程中断时可能会抛出 `InterruptedException`

### 3.2.2 释放资源

释放资源相比于获取资源来说，会简单许多。在AQS中只有一小段实现。先释放锁,成功再唤醒后继线程

源码：

```

public final boolean release(int arg) {
    // 先调用具体实现类的释放规则
    if (tryRelease(arg)) {
        Node h = head;
        if (h != null && h.waitStatus != 0)
            unparkSuccessor(h);
        return true;
    }
    return false;
}

private void unparkSuccessor(Node node) {
    // 如果状态是负数，尝试把它设置为0
    int ws = node.waitStatus;
    if (ws < 0)
        compareAndSetWaitStatus(node, ws, 0);
    // 得到头结点的后继结点head.next
    Node s = node.next;
    // 如果这个后继结点为空或者状态大于0
    // 通过前面的定义我们知道，大于0只有一种可能，就是这个结点已被取消
    if (s == null || s.waitStatus > 0) {
        s = null;
        // 等待队列中所有还有用的结点，都向前移动
        for (Node t = tail; t != null && t != node; t = t.prev)
            if (t.waitStatus <= 0)
                s = t;
    }
    // 如果后继结点不为空，
    if (s != null)
        LockSupport.unpark(s.thread);
}

```

### 3.2.3 超时释放资源

通过调用同步器的 `doAcquireNanos(int arg, long nanosTimeout)` 方法可以超时获取同步状态，即在指定的时间段内获取同步状态，如果获取到同步状态则返回 `true`，否则，返回 `false`。

超时具体实现: 先拿到截止时间, 然后在自旋里看是否到了截止时间, 如果到了就算超时



```
// java.util.concurrent.locks.AbstractQueuedSynchronizer#doAcquireNanos
/**
 * The number of nanoseconds for which it is faster to spin
 * rather than to use timed park. A rough estimate suffices
 * to improve responsiveness with very short timeouts.
 */
static final long spinForTimeoutThreshold = 1000L;
private boolean doAcquireNanos(int arg, long nanosTimeout) throws InterruptedException
{
    if (nanosTimeout <= 0L)
        return false;
    final long deadline = System.nanoTime() + nanosTimeout;
    final Node node = addWaiter(Node.EXCLUSIVE);
    try {
        for (;;) {
            final Node p = node.predecessor();
            if (p == head && tryAcquire(arg)) {
                setHead(node);
                p.next = null; // help GC
                return true;
            }
            nanosTimeout = deadline - System.nanoTime();
            if (nanosTimeout <= 0L) {
                cancelAcquire(node);
                return false;
            }
            if (shouldParkAfterFailedAcquire(p, node) && nanosTimeout >
                SPIN_FOR_TIMEOUT_THRESHOLD)
                LockSupport.parkNanos(this, nanosTimeout);
            if (Thread.interrupted())
                throw new InterruptedException();
        }
    } catch (Throwable t) {
        cancelAcquire(node);
        throw t;
    }
}
```

如果 nanosTimeout 小于等于 spinForTimeoutThreshold (1000 纳秒) 时, 将不会使该线程进行超时等待, 而是进入快速的自旋过程。原因在于, 非常短的超时等待无法做到十分精确, 如果这时再进行超时等待, 相反会让 nanosTimeout 的超时从整体上表现得反而不精确。因此, 在超时非常短的场景下, 同步器会进入无条件的快速自旋

### 3.3 ReentrantLock (重入锁)

它有几个重要的功能,

- 公平锁和非公平锁
- 可重入

默认是非公平锁, 毕竟性能高嘛

可重入性是用到了AQS中状态标识字段,其值表示重入次数

## 获取锁

1. CAS操作抢占锁, 抢占成功则修改锁的状态为1, 将线程信息记录到锁当中, 返回state=1
2. 抢占不成功, tryAcquire获取锁资源, 获取成功直接返回, 获取不成功, 新建一个节点插入到当前AQS队列的尾部, acquireQueued (node) 表示唤醒AQS队列中的节点再次去获取锁

## 释放锁

1. 获取锁的状态值, 释放锁将状态值-1
2. 判断当前释放锁的线程和锁中保存的线程信息是否一致, 不一致会抛出异常
3. 状态只-1直到为0, 锁状态值为0表示不再占用, 为空闲状态

[ReentrantLock详解\\_SunStaday的博客-CSDN博客\\_reentrantlock](#)

其实它自己没什么功能, 只是实现了AQS

## 3.4 ReentrantReadWriteLock (读写锁)

ReentrantReadWriteLock是Lock的另一种实现方式, 我们已经知道了ReentrantLock是一个排他锁, 同一时间只允许一个线程访问, 而ReentrantReadWriteLock允许多个读线程同时访问, 但不允许写线程和读线程、写线程和写线程同时访问(称为写饥饿)。相对于排他锁, 提高了并发性。在实际应用中, 大部分情况下对共享数据(如缓存)的访问都是读操作远多于写操作, 这时ReentrantReadWriteLock能够提供比排他锁更好的并发性和吞吐量。

读写锁内部维护了两个锁, 一个用于读操作, 一个用于写操作。所有ReadWriteLock实现都必须保证writeLock操作的内存同步效果也要保持与相关readLock的联系。也就是说, 成功获取读锁的线程会看到写入锁之前版本所做的所有更新。

ReentrantReadWriteLock支持以下功能:

- 1) 支持公平和非公平的获取锁的方式;
- 2) 支持可重入。读线程在获取了读锁后还可以获取读锁; 写线程在获取了写锁之后既可以再次获取写锁又可以获取读锁;

3) 还允许从写入锁降级为读取锁，其实现方式是：先获取写入锁，然后获取读取锁，最后释放写入锁。但是，从读取锁升级到写入锁是不允许的；

4) 读取锁和写入锁都支持锁获取期间的中断；

5) Condition支持。仅写入锁提供了一个 Condition 实现；读取锁不支持 Condition，readLock().newCondition() 会抛出 UnsupportedOperationException。

```
ReentrantReadWriteLock readWriteLock = new ReentrantReadWriteLock();
ReentrantReadWriteLock.WriteLock writeLock = readWriteLock.writeLock();
writeLock.lock();
// todo do some thing
writeLock.unlock();

ReentrantReadWriteLock.ReadLock readLock = readWriteLock.readLock();
readLock.lock();
// todo do some thing
readLock.unlock();
```

ReentrantReadWriteLock实现了读写锁，但它有一个小弊端，就是在“写”操作的时候，其它线程不能写也不能读。我们称这种现象为“**写饥饿**”

**导致写线程饥饿的情况:** 当线程 A 持有读锁读取数据时，线程 B 要获取写锁修改数据就只能到队列里排队。此时又来了线程 C 读取数据，那么线程 C 就可以获取到读锁，而要执行写操作线程 B 就要等线程 C 释放读锁。由于该场景下读操作远远大于写的操作，此时可能会有很多线程来读取数据而获取到读锁，那么要获取写锁的线程 B 就只能一直等待下去，最终导致饥饿。

[读写锁饥饿问题解决方案之StampedLock | 码农家园 \(codenong.com\)](#)

[14 锁接口和类 · 深入浅出Java多线程 \(redspider.group\)](#)

[ReentrantReadWriteLock用法 - 简书 \(jianshu.com\)](#)

## 3.5 StampedLock

StampedLock 类是在Java 8 才发布的,它没有实现Lock接口和ReadWriteLock接口，但它其实是实现了“读写锁”的功能，并且性能比ReentrantReadWriteLock更高。StampedLock还把读锁分为了“乐观读锁”和“悲观读锁”两种。StampedLock的底层并不是基于AQS的。

### 1 优点

1. 没有饥饿发生
2. 支持锁升级、

3. 锁降级、
4. 超时和中断
5. 一次唤醒所有读节点

## 2 缺点

1. 无condition
2. 不公平
3. 写不可重入

前面提到了ReentrantReadWriteLock会发生“写饥饿”的现象，但StampedLock不会。它是怎么做到的呢？它的核心思想在于，**在读的时候如果发生了写，应该通过重试的方式来获取新的值，而不应该阻塞写操作。这种模式也就是典型的无锁编程思想，和CAS自旋的思想一样**

个人感觉: 因为它引入了乐观锁机制, 当加了乐观读锁时还是可以拥有写锁的, 这样就避免的写饥饿, 能否避免, 还是得看使用者, 如果直接使用悲观读锁应该也会有写饥饿

官方使用案例:

```

public class Point {
    private final StampedLock stampedLock = new StampedLock();

    private double x;
    private double y;

    public void move(double deltaX, double deltaY) {
        long stamp = stampedLock.writeLock(); // 获取写锁
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            stampedLock.unlockWrite(stamp); // 释放写锁
        }
    }

    public double distanceFromOrigin() {
        long stamp = stampedLock.tryOptimisticRead(); // 获得一个乐观读锁
        // 注意下面两行代码不是原子操作
        // 假设x,y = (100,200)
        double currentX = x;
        // 此处已读取到x=100, 但x,y可能被写线程修改为(300,400)
        double currentY = y;
        // 此处已读取到y, 如果没有写入, 读取是正确的(100,200)
        // 如果有写入, 读取是错误的(100,400)
        if (!stampedLock.validate(stamp)) { // 检查乐观读锁后是否有其他写锁发生
            stamp = stampedLock.readLock(); // 获取一个悲观读锁
            try {
                currentX = x;
                currentY = y;
            } finally {
                stampedLock.unlockRead(stamp); // 释放悲观读锁
            }
        }
        return Math.sqrt(currentX * currentX + currentY * currentY);
    }
}
//悲观读锁以及读锁升级写锁的使用
void moveIfAtOrigin(double newX, double newY) {

    long stamp = stampedLock.readLock(); //悲观读锁
    try {
        while (x == 0.0 && y == 0.0) {
            long ws = stampedLock.tryConvertToWriteLock(stamp); //读锁转换为写锁
            if (ws != 0L) { //转换成功

                stamp = ws; //票据更新
                x = newX;
                y = newY;
                break;
            } else {

```

```

        stampedLock.unlockRead(stamp); //转换失败释放读锁
        stamp = stampedLock.writeLock(); //强制获取写锁
    }
}
} finally {
    stampedLock.unlock(stamp); //释放所有锁
}
}
}
}

```

[Java并发 \(8\) - 读写锁中的性能之王: StampedLock - knock\\_小新 - 博客园 \(cnblogs.com\)](#)

[StampedLock实现原理\\_曲终人散-CSDN博客](#)

[StampedLock详解\\_张孟浩\\_jay的博客-CSDN博客\\_stampedlock](#)

[读写锁饥饿问题解决方案之StampedLock\\_breakout\\_alex的博客-CSDN博客\\_读写锁写饥饿](#)

## 3.6 Condition 接口

Condition的作用用一句话概括就是为了实现线程的等待（await）和唤醒（signal），多线程情况下为什么需要等待唤醒机制？原因是有些线程执行到某个阶段需要等待符合某个条件才可以继续执行，

有一个经典的场景就是在容量有限的缓冲区实现生产者消费者模型，如果缓冲区满了，这个时候生产者就不能再生产了，就要阻塞等待消费者消费，当缓冲区为空了，消费者就要阻塞等待生产者生产。

案例如下：

```

public class MyTest {
    Lock lock = new ReentrantLock();
    Condition condition2 = lock.newCondition();
    Condition condition1 = lock.newCondition();

    public static void main(String[] args) {
        MyTest test = new MyTest();

        new Thread(()->{
            try {
                test.awaitTest();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }).start();

        new Thread(test::signalTest).start();
    }

    public void awaitTest() throws Exception{
        try {
            lock.lock();
            System.out.println("等待中");
            condition1.await();
            System.out.println("解除等待");
        } finally {
            lock.unlock();
        }
    }

    public void signalTest(){
        try {
            lock.lock();
            condition1.signal();
            System.out.println("继续执行");
        } finally {
            lock.unlock();
        }
    }
}

```

结果如下:

等待中

继续执行

解除等待

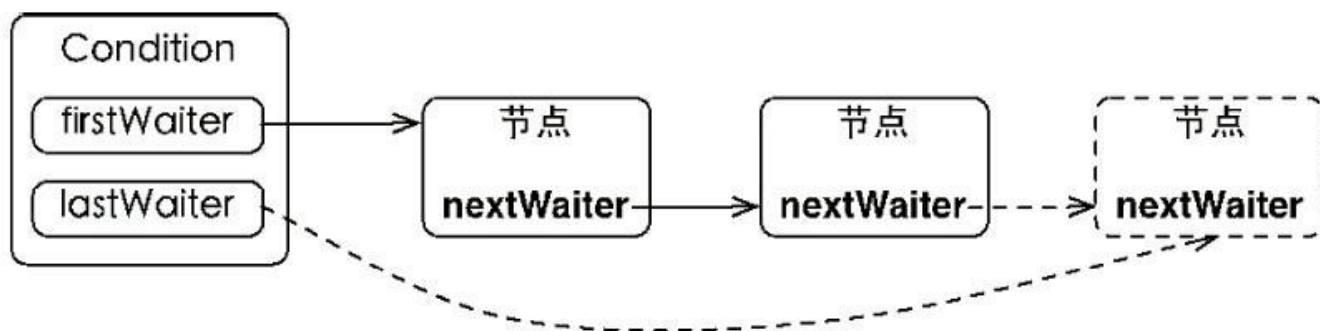
## 1. 等待队列

等待队列是一个 FIFO 的队列，在队列中的每个节点都包含了一个线程引用，该线程就是在 Condition 对象上等待的线程，`Condition condition = new ReentrantLock().newCondition();`，每次调用 `newCondition()` 就会生成一个等待队列。

如果一个线程调用了 `Condition.await()` 方法，那么该线程将会释放锁、构造成节点加入等待队列并进入等待状态。

这些节点复用了同步器中节点的定义，所以同步队列和等待队列中节点类型都是同步器的静态内部类 `AbstractQueuedSynchronizer.Node`。

一个 Condition 包含一个等待队列，Condition 拥有首节点（`firstWaiter`）和尾节点（`lastWaiter`）。当前线程调用 `Condition.await()` 方法，将会以当前线程构造节点，并将节点从尾部加入等待队列，如图

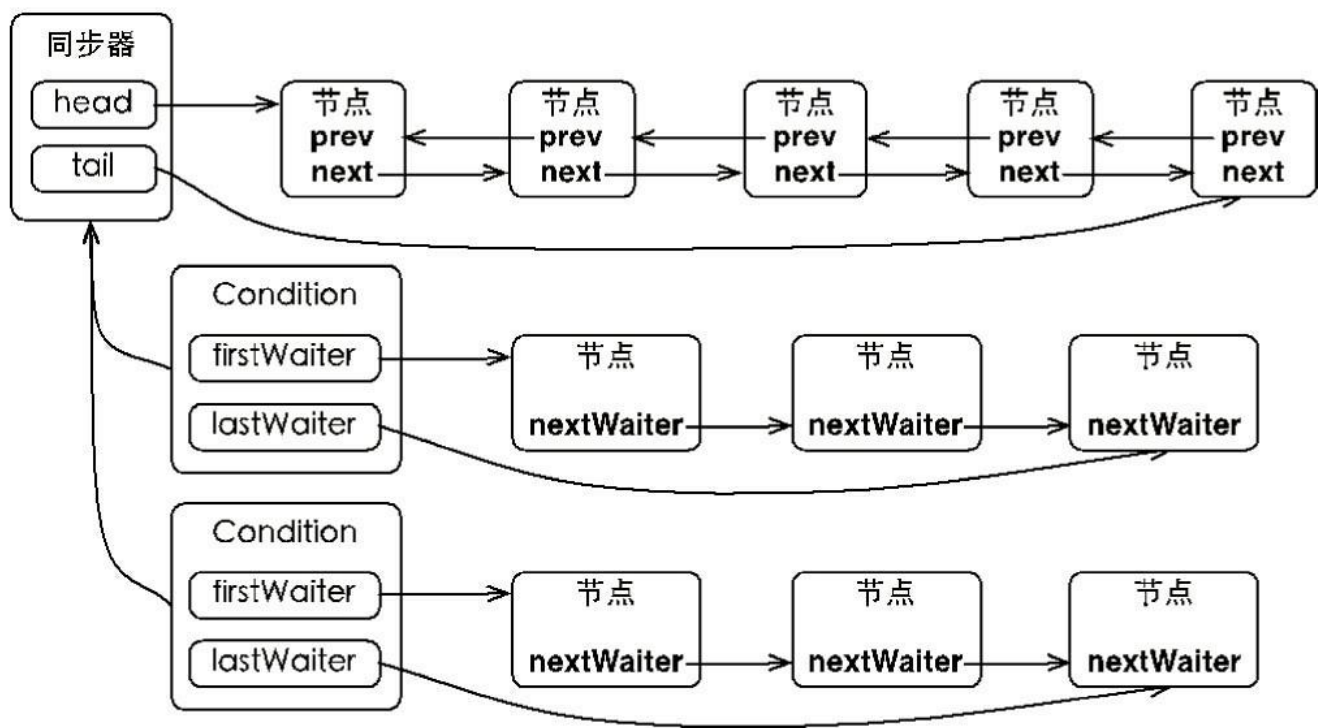


Condition 拥有首尾节点的引用，而新增节点只需要将原有的尾节点 `nextWaiter` 指向它，并且更新尾节点即可。（这和AQS的同步队列是一样的）

上述节点引用更新的过程并没有使用 CAS 保证，原因在于调用 `await()` 方法的线程必定是获取了锁的线程，也就是说该过程是由锁来保证线程安全的

在 Object 的监视器模型上，一个对象拥有一个同步队列和等待队列，而并发包中的 `Lock`（更确切地说是同步器）拥有一个同步队列和多个等待队列，如图





## 2. 等待

调用 Condition 的 `await()` 方法（或者以 `await` 开头的方法），会使当前线程进入等待队列并释放锁，同时线程状态变为等待状态。当从 `await()` 方法返回时，当前线程一定获取了 Condition 相关的锁，源码如下

```

// java.util.concurrent.locks.AbstractQueuedSynchronizer.ConditionObject#await()
public final void await() throws InterruptedException {
    if (Thread.interrupted())
        throw new InterruptedException();
    // 当前线程加入等待队列
    Node node = addConditionWaiter();
    // 释放同步状态，也就是释放锁（因为这是await嘛，释放资源的暂停，这样同步队列中后面的
    节点就能执行了）
    int savedState = fullyRelease(node);
    int interruptMode = 0;
    while (!isOnSyncQueue(node)) {
        LockSupport.park(this);
        if ((interruptMode = checkInterruptWhileWaiting(node)) != 0)
            break;
    }
    if (acquireQueued(node, savedState) && interruptMode != THROW_IE)
        interruptMode = REINTERRUPT;
    if (node.nextWaiter != null) // clean up if cancelled
        unlinkCancelledWaiters();
    if (interruptMode != 0)
        reportInterruptAfterWait(interruptMode);
}

```

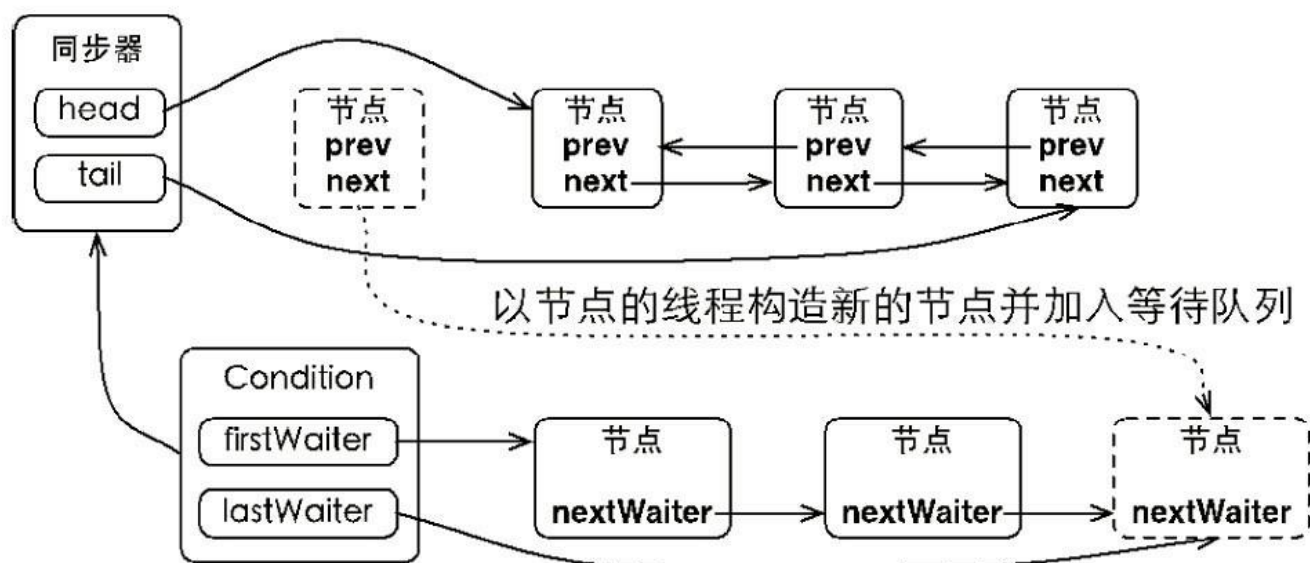
调用该方法的线程成功获取了锁的线程，也就是同步队列中的首节点，该方法会将当前线程构造成节点并加入等待队列中，然后释放同步状态，唤醒同步队列中的后继节点，然后当前线程会进入等待状态。

如果从队列的角度去看，当前线程加入 Condition 的等待队列，同步队列的首节点并不会直接加入等待队列，而是通过 `addConditionWaiter()` 方法把当前线程构造成一个新的节点并将其加入等待队列中。源码如下：

```

private Node addConditionWaiter() {
    Node t = lastWaiter;
    // If lastWaiter is cancelled, clean out.
    if (t != null && t.waitStatus != Node.CONDITION) {
        unlinkCancelledWaiters();
        t = lastWaiter;
    }
    Node node = new Node(Thread.currentThread(), Node.CONDITION);
    if (t == null)
        firstWaiter = node;
    else
        t.nextWaiter = node;
    lastWaiter = node;
    return node;
}

```



总结: 要使用condition, 则必须获得锁, (当前线程获得锁了,表示在同步队列的首位了), 此时使用await方法就会把这个节点放到等待队列中(还会释放锁)

然后等待队列中的节点排队等待被唤醒

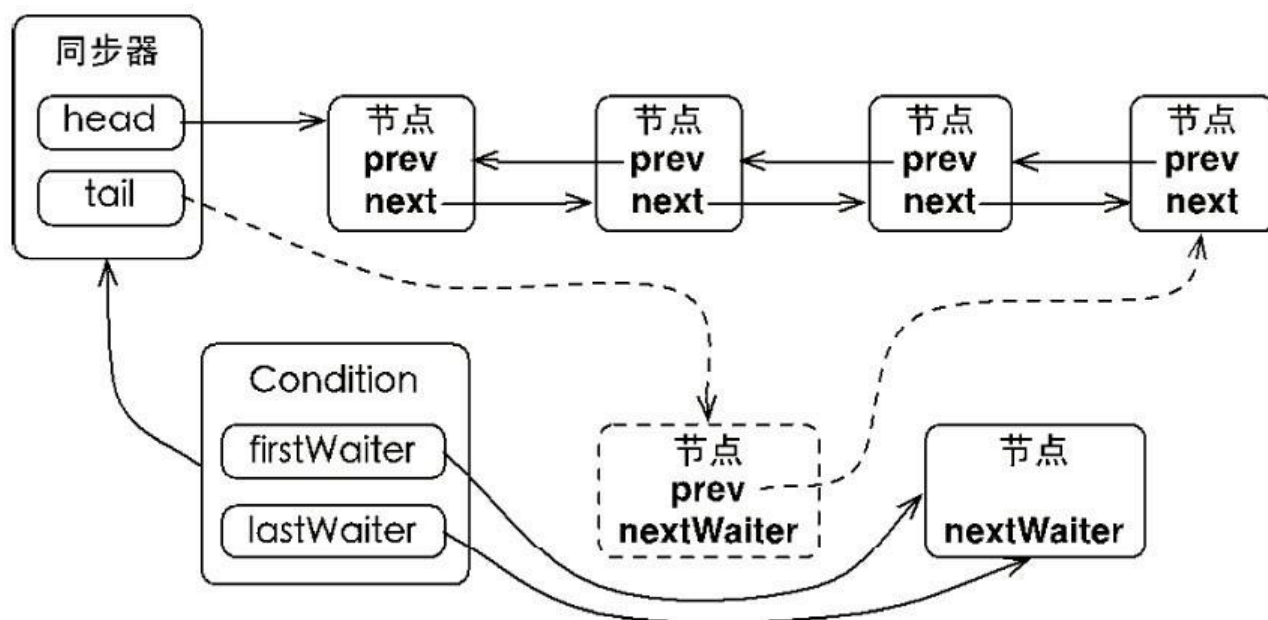
### 3. 通知

调用 Condition 的 signal()方法, 将会唤醒在等待队列中等待时间最长的节点(首节点), 在唤醒节点之前, 会将节点移到同步队列中。Condition 的 signal()方法, 源码如下:

```
public final void signal() {
    if (!isHeldExclusively())
        throw new IllegalMonitorStateException();
    Node first = firstWaiter;
    if (first != null)
        doSignal(first);
}
```

调用该方法的前置条件是当前线程必须获取了锁, 可以看到 signal()方法进行了 isHeldExclusively()检查, 也就是当前线程必须是获取了锁的线程。

接着获取等待队列的首节点, 将其移动到同步队列并使用 LockSupport 唤醒节点中的线程。节点从等待队列移动到同步队列的过程如图



通过调用同步器的 `enq(Node node)` 方法，等待队列中的头节点线程安全地移动到同步队列。当节点移动到同步队列后，当前线程再使用 `LockSupport` 唤醒该节点的线程。

被唤醒后的线程，将从 `await()` 方法中的 `while` 循环中退出（`isOnSyncQueue(Nodenode)` 方法返回 `true`，节点已经在同步队列中），进而调用同步器的 `acquireQueued()` 方法加入到获取同步状态的竞争中。

成功获取同步状态（或者说锁）之后，被唤醒的线程将从先前调用的 `await()` 方法返回，此时该线程已经成功地获取了锁。

总结: 要想唤醒线程, 先得获取锁(也就是说, 只有等待队列中的首位才有操作权限), 唤醒后, 就会把节点加入到同步队列中,

在同步队列中等待获得锁

[AQS-Condition介绍 - 猿起缘灭 - 博客园 \(cnblogs.com\)](#)

[AbstractQueuedSynchronizer深入理解 - 博客园 \(cnblogs.com\)](#)

[图解AQS的设计与实现 - Java填坑笔记 - 博客园 \(cnblogs.com\)](#)

[AQS\(AbstractQueuedSynchronizer\)源码深度解析\(3\)—同步队列以及独占式获取锁、释放锁的原理【一万字】\\_刘Java-CSDN博客](#)

《Java并发编程艺术》

[11 AQS · 深入浅出Java多线程 \(redspider.group\)](#)

[并发工具（锁）：深入Lock+Condition - 知乎 \(zhihu.com\)](#)